DOSSIER TECHNIQUE ET FONCTIONNEL

COLLAB EDITOR

Conception, architecture et validation

Application collaborative de rédaction d’un journal étudiant

Nature du livrable	Rapport de conception et architecture AWS
Périmètre	Besoins, UML, technologies, architecture cible et réelle
Date	26 octobre 2026
Statut	Version de dépôt

Les fonctionnalités projetées sont distinguées de celles effectivement déployées et testées.

Table des matières

Résumé exécutif . . . 3
1. Contexte, problématique et démarche . . . 4
1.1 Contexte et objectifs . . . 4
1.2 Problématique technique . . . 4
1.3 Méthodologie de conception . . . 4
2. Analyse fonctionnelle et exigences . . . 5
2.1 Acteurs et périmètre . . . 5
2.2 Exigences fonctionnelles . . . 5
2.3 Exigences non fonctionnelles et critères d'acceptation . . . 5
3. État de l'art : édition collaborative . . . 8
3.1 Pourquoi la synchronisation naïve ne suffit pas . . . 8
3.2 Operational Transformation (OT) . . . 8
3.3 Conflict-free Replicated Data Types (CRDT) . . . 8
3.4 Comparaison OT / CRDT . . . 8
3.5 Google Docs comme référence de conception . . . 9
3.6 Décision : ShareDB et OT . . . 9
3.7 Exemple de concurrence et transformation OT . . . 9
4. Architecture applicative et répartition des responsabilités . . . 10
4.1 Découpage logique . . . 10
4.2 Répartition de la charge et évolutivité . . . 10
4.3 Frontières de confiance et responsabilités . . . 10
5. Conception de la collaboration en temps réel . . . 11
5.1 Cycle d'une modification . . . 11
5.2 Reconnexion et historique . . . 11
5.3 Présence et curseurs . . . 11
6. Modèle de données et invariants . . . 12
7. Conception des règles métier concurrentes . . . 14
7.1 Crédit quotidien et prévention des contournements . . . 14
7.2 Publication vivante et quota PDF . . . 14
7.3 Erreurs et défaillances . . . 14

7.3.1 Déroulement transactionnel du téléchargement 14
7.4 Outils, technologies et bibliothèques : rôle dans la solution 14
8. Déploiement et architecture AWS 17
8.1 Architecture cible 17
8.2 État de déploiement vérifié au 26 octobre 2026 19
8.3 Difficultés rencontrées et corrections 19
8.4 Limites et risques résiduels 20
9. Stratégie de validation 21
10. Discussion critique et perspectives 22
Conclusion 23
Références et éléments à consolider 24
11. État de validation et résultats mesurés 25
11.1 Campagne fonctionnelle Playwright 25
11.2 Concurrence des téléchargements PDF 25
11.3 Limites et perspectives 25
Annexe A – Cas d’utilisation et descriptions textuelles 26
Tableau de couverture des cas d’utilisation 26
Annexe B – Technologies utilisées et justification des choix 35
Annexe C – Limites, cohérence sémantique et perspectives 36
Conclusion générale 37

Étude des approches collaboratives, modélisation, choix architecturaux, cohérence transactionnelle et déploiement AWS

Projet : journal étudiant collaboratif Version de dépôt – 26 octobre 2026 Statut : architecture cible distinguée du déploiement réel ; validation partielle documentée.

Résumé exécutif

Collab Editor est une application web de rédaction collaborative destinée à la préparation d'un journal étudiant. Elle associe édition simultanée, présence des rédacteurs, historique et restauration, publication d'un lien stable vers le document courant, génération de PDF et limitation stricte des téléchargements. Le projet comporte également un crédit quotidien de 10 000 caractères par rédacteur et des mécanismes de limitation des comportements abusifs.

La difficulté principale n'est pas l'interface d'édition : elle réside dans la coordination d'écritures concurrentes, la préservation des opérations acceptées, l'application de règles métier partagées entre sessions et la gestion de téléchargements concurrents sans dépassement de quota. Ce dossier expose les alternatives étudiées, les arbitrages et les invariants retenus. Il sépare explicitement conception cible, éléments implémentés et éléments vérifiés sur AWS.

1. Contexte, problématique et démarche

1.1 Contexte et objectifs

La rédaction collective d'un journal implique des auteurs qui interviennent simultanément, parfois sur les mêmes paragraphes. Une solution reposant sur l'échange de fichiers entraîne des conflits de versions, des écrasements de modifications et une traçabilité insuffisante. Collab Editor cherche à fournir un état commun du document, tout en encadrant sa publication et sa diffusion.

1.2 Problématique technique

Comment garantir que plusieurs rédacteurs puissent travailler sur un document partagé avec convergence des contenus et conservation des modifications acceptées, tout en appliquant des contraintes transactionnelles sur les crédits d'écriture et les quotas de téléchargement, dans une architecture exploitable localement et sur AWS ?

1.3 Méthodologie de conception

La démarche combine analyse des besoins, étude comparative des mécanismes de collaboration, définition d'invariants, découpage en composants, modélisation des données, scénarios de concurrence, implémentation incrémentale, tests et préparation du déploiement. Les choix ne sont pas présentés comme des vérités universelles : ils sont appréciés au regard du périmètre du projet, du temps de réalisation et des risques de cohérence.

2. Analyse fonctionnelle et exigences

2.1 Acteurs et périmètre

Acteur	Responsabilités et interactions
Rédacteur	Déclare un nom et un e-mail ; crée, ouvre, modifie et restaure des documents ; publie des liens.
Collaborateur	Accède à un document partagé ; édite simultanément ; observe la présence et les positions.
Visiteur	Accède à un lien de publication et demande un PDF dans la limite du quota.
Responsable / supervision	Consulte les documents ouverts, les liens actifs, les quotas et les blocages.
Services externes	Infrastructure AWS, stockage privé et notification e-mail selon configuration.

2.2 Exigences fonctionnelles

— Identification déclarative par nom et e-mail, sans prétendre vérifier la propriété de l'adresse.

— Création et partage de documents, édition simultanée, présence et positions des rédacteurs.

— Historique des versions, consultation et restauration non destructive.

— Lien de publication stable avec quota ; PDF représentant une révision cohérente du document courant.

— Crédit quotidien de 10 000 caractères, détection de rejets répétés et blocage temporaire.

— Supervision des documents, liens, quotas et blocages ; notification lors de l'épuisement du dernier téléchargement.

2.3 Exigences non fonctionnelles et critères d'acceptation

Propriété	Invariant / critère
Convergence	Les clients synchronisés finissent par observer un même état après traitement des opérations acceptées.
Durabilité	Aucune opération confirmée comme acceptée ne doit être perdue.
Quota PDF	Deux demandes simultanées ne doivent jamais distribuer plus d'exemplaires que le quota.
Crédit	Deux sessions d'un même rédacteur ne peuvent contourner le plafond quotidien.

Traçabilité	Une restauration conserve l'historique et crée un nouvel état courant.
Sécurité	Secrets hors frontend ; accès S3 privé ; liaisons chiffrées selon le contexte.



Figure 1 – Vue de conception du système

ID	Cas d'utilisation	Acteur principal	Résultat attendu
UC-01	S'identifier	Rédacteur	Session déclarative ouverte
UC-02	Créer et partager un document	Rédacteur	Document créé et accès partagé
UC-03	Éditer simultanément	Rédacteurs	Révisions convergentes et persistées
UC-04	Voir présence et curseurs	Rédacteur	Collaborateurs visibles en temps réel

UC-05	Consulter l’historique	Rédacteur	Versions antérieures consultables
UC-06	Restaurer une version	Rédacteur	Nouvelle révision courante conservant l’historique
UC-07	Créer un lien de publication	Rédacteur	Lien stable avec quota défini
UC-08	Télécharger un PDF	Visiteur	PDF courant remis si quota disponible
UC-09	Gérer le crédit d’écriture	Système / rédacteur	Consommation et blocage cohérents
UC-10	Consulter la supervision	Responsable	État des documents, liens et blocages
UC-11	Notifier l’épuisement du quota	Système	Notification au créateur selon configuration

3. État de l’art : édition collaborative

3.1 Pourquoi la synchronisation naïve ne suffit pas

Une stratégie « dernier enregistrement gagnant » remplace l’état complet du document par la dernière copie reçue. Deux auteurs travaillant simultanément peuvent alors perdre les modifications de l’autre. Une stratégie de verrouillage exclusif évite certaines collisions, mais dégrade fortement l’expérience de coédition. Les systèmes collaboratifs modernes représentent donc généralement les modifications sous forme d’opérations ou de structures répliquées, avec des règles de fusion explicites.

3.2 Operational Transformation (OT)

L’OT représente une modification comme une opération appliquée à une révision donnée. Lorsqu’une opération concurrente a déjà été intégrée, l’opération entrante est transformée par rapport aux changements intervenus afin de préserver l’intention des utilisateurs autant que le modèle le permet. Une autorité centrale peut ordonner les révisions, valider les opérations et diffuser leur résultat. La correction dépend de la définition des opérations, de leur transformation et du protocole de synchronisation ; l’OT n’est pas automatiquement exempt de conflits.

Exemple conceptuel : A insère « X » en position 2 et B insère « Y » en position 2 sur la même révision. Un ordre déterministe et une transformation des positions permettent d’obtenir un document identique chez les deux auteurs, plutôt que de perdre une insertion.

3.3 Conflict-free Replicated Data Types (CRDT)

Les CRDT sont des structures de données conçues pour permettre la fusion déterministe de modifications concurrentes sous certaines hypothèses. Elles sont particulièrement pertinentes pour les usages décentralisés ou hors ligne, car les répliques peuvent progresser puis converger sans ordonnancement central de chaque opération. En contrepartie, les CRDT de texte introduisent des identifiants, des métadonnées et des considérations de compactage, de causalité et de gestion des suppressions. Leur coût dépend fortement de l’implémentation et de la charge.

3.4 Comparaison OT / CRDT

Critère	OT	CRDT
Coordination	Souvent serveur autoritaire et révisions ordonnées	Fusion entre répliques ; coordination centrale non intrinsèquement nécessaire
Hors ligne	Possible, mais réconciliation et protocole à concevoir	Souvent favorable aux éditions déconnectées
Intégration métier	Validation centralisée naturelle avant acceptation	Validation métier globale à articuler avec la fusion
Complexité	Transformations et gestion de versions délicates	Métadonnées, causalité, nettoyage et modèle documentaire
Pertinence ici	Bonne adéquation avec backend central et ShareDB	Alternative crédible, mais migration et validation supplémentaires

3.5 Google Docs comme référence de conception

Google Docs constitue un exemple d'expérience de coédition : présence, modifications visibles rapidement, reprise de connexion et historique. Il est pertinent d'en analyser les comportements observables et les principes publiquement documentés, mais il serait incorrect d'affirmer connaître l'architecture interne exacte ou l'algorithme actuellement utilisé par Google. L'enseignement utile pour Collab Editor est la séparation entre expérience utilisateur, protocole de synchronisation, conservation des versions et règles d'autorisation.

La comparaison avec Google Docs a servi de référence fonctionnelle, non de preuve qu'une architecture identique a été reproduite. Collab Editor retient une approche plus réduite et maîtrisable, adaptée à son périmètre académique.

3.6 Décision : ShareDB et OT

Le choix de ShareDB est cohérent avec une architecture Node.js centralisée et un besoin de documents partagés synchronisés par WebSocket. ShareDB apporte des primitives de souscription, de versions et de traitement d'opérations, ce qui réduit le volume de protocole à développer. Les contraintes métier restent de la responsabilité des services applicatifs : l'acceptation d'une opération doit respecter les droits, le crédit et la persistance. Ce choix ne signifie pas que CRDT serait techniquement inférieur ; il constitue un arbitrage d'intégration, de maîtrise des risques et de délai.

3.7 Exemple de concurrence et transformation OT

Étape	**Situation**	**Résultat attendu**
hline 1	Clients A et B à la révision R10	Les deux clients partagent le même état initial.
2	A et B soumettent des opérations concurrentes	Le serveur sérialise les acceptations et transforme les opérations compatibles.
3	Acceptation de R11 puis R12	Les opérations acceptées sont durables et diffusées.
4	Clients synchronisés	Les clients convergent vers le même texte et la même révision.

4. Architecture applicative et répartition des responsabilités

4.1 Découpage logique

L'interface React/TypeScript prend en charge l'édition, la présence, l'historique, la publication et les messages d'erreur. L'API Fastify expose les opérations HTTP, contrôle les entrées et orchestre les services. ShareDB traite les opérations collaboratives et leur diffusion. PostgreSQL constitue la source de vérité transactionnelle pour les documents, les versions, les crédits et les quotas. Redis/Valkey sert à la coordination et aux données éphémères, sans remplacer la persistance. La génération PDF est isolée de son stockage : local en développement, S3 privé en production.

4.2 Répartition de la charge et évolutivité

La séparation frontend statique / API / stockage permet d'adapter chaque partie indépendamment. CloudFront peut distribuer les fichiers statiques sans solliciter Node.js. Les sessions WebSocket consomment des connexions durables sur l'API ; l'augmentation du nombre d'instances nécessite un mécanisme de coordination et une stratégie de routage compatibles avec ShareDB. Redis peut contribuer à cette coordination, mais sa seule présence ne démontre pas qu'un déploiement multi-instance est opérationnel. Les écritures critiques restent arbitrées par PostgreSQL.

4.3 Frontières de confiance et responsabilités

Le navigateur n'est pas une source d'autorité pour les crédits ou les quotas. Les vérifications métier s'exécutent côté serveur. Le bucket PDF reste privé ; les autorisations IAM de l'instance déterminent les opérations S3 permises. Les secrets de base de données doivent être obtenus côté backend via les mécanismes AWS appropriés et ne doivent jamais être intégrés aux variables publiques Vite.

5. Conception de la collaboration en temps réel

5.1 Cycle d’une modification

Le client édite une révision connue et soumet une opération via WebSocket. Le serveur identifie le document et le rédacteur, applique les contrôles d’accès et de crédit, traite la concurrence et persiste l’opération acceptée. La nouvelle révision est ensuite communiquée aux sessions abonnées. En cas de refus, le client reçoit une réponse explicite et doit réconcilier son affichage avec l’état serveur ; un simple changement visuel local ne vaut pas confirmation durable.

5.2 Reconnexion et historique

Une reconnexion impose de distinguer modifications locales en attente, opérations déjà confirmées et révision serveur actuelle. Le mécanisme de resynchronisation doit éviter de soumettre deux fois une même opération et préserver l’ordre des révisions. La restauration d’une ancienne version doit créer une nouvelle version courante, sans effacer l’historique antérieur.

5.3 Présence et curseurs

Les indicateurs de présence et les positions des curseurs relèvent d’un état transitoire. Leur absence temporaire n’implique pas la perte du document. La gestion des déconnexions et des sessions expirées évite d’afficher indéfiniment des rédacteurs fantômes.

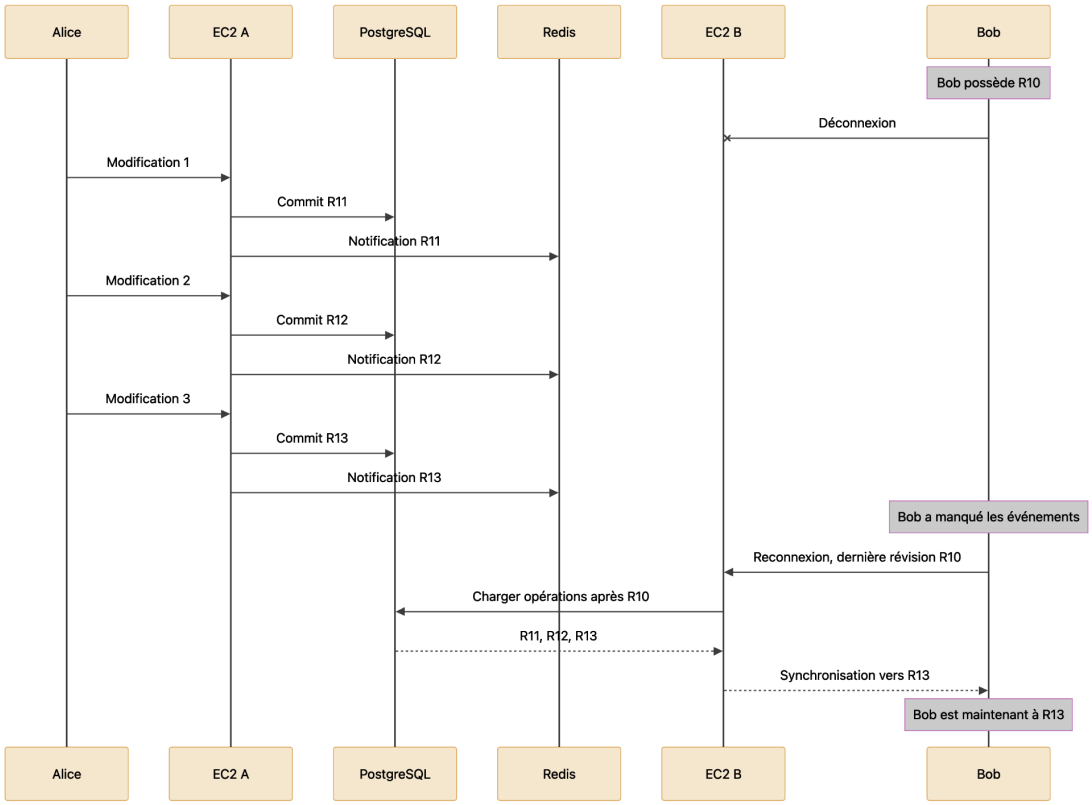


Figure 2 – Séquence de collaboration multi-instance (scénario cible).

6. Modèle de données et invariants

Le modèle conceptuel distingue les entités Editor, Document, DocumentAccess, DocumentVersion, EditOperation, DailyCredit, EditorBlock, DownloadLink et PdfArtifact. Selon la stratégie implémentée, DownloadReservation et Notification complètent le suivi des téléchargements et alertes. Ce modèle présente les principales entités et leurs relations fonctionnelles, indépendamment des détails de stockage propres à chaque table.

Entité	Rôle
Document / DocumentAccess	Contenu partagé, propriété et autorisations.
DocumentVersion / EditOperation	Révisions, opérations acceptées et traçabilité.
DailyCredit / EditorBlock	Consommation par jour, compteurs de rejets et suspension.
DownloadLink	Lien stable, limite et consommation de téléchargements.
PdfArtifact	PDF produit pour une révision déterminée, stocké localement ou dans S3.
DownloadReservation / Notification	Réservation transitoire et suivi de notification, si activés dans l'implémentation.

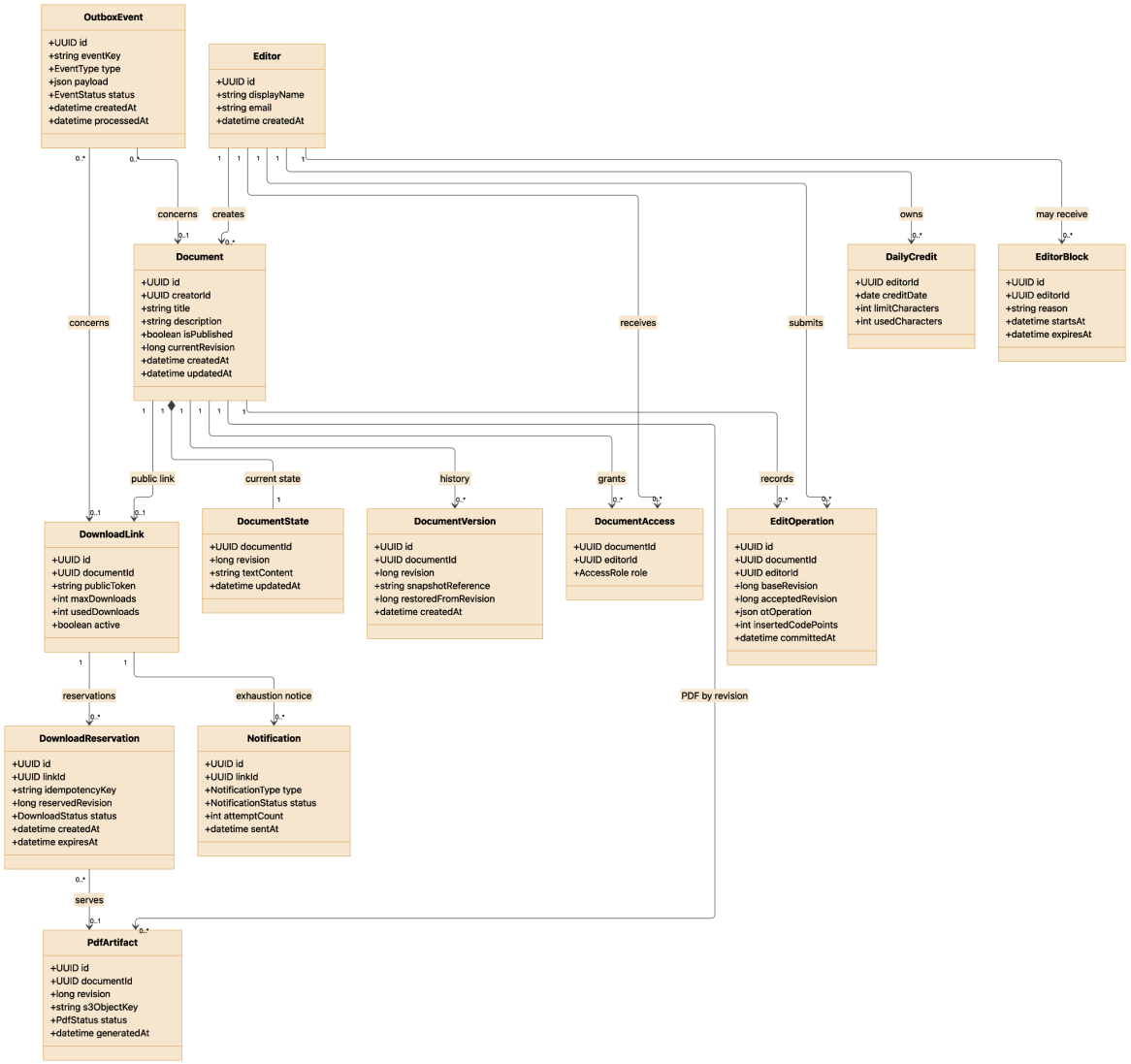


Figure 3 – Diagramme de classes du système Collab Editor.

7. Conception des règles métier concurrentes

7.1 Crédit quotidien et prévention des contournements

Le plafond de 10 000 caractères est attaché au rédacteur et à une période journalière, non à un onglet. Une vérification uniquement côté navigateur serait contournable. La consommation doit être validée transactionnellement et de manière concurrente : deux sessions ne peuvent accepter simultanément des écritures qui, additionnées, dépassent le plafond. Les refus doivent être traçables afin d'appliquer les règles de blocage temporaire définies par le métier.

7.2 Publication vivante et quota PDF

Un lien public pointe vers le document vivant : deux téléchargements effectués à des moments différents peuvent refléter des révisions différentes. Pour chaque demande autorisée, le serveur sélectionne une révision cohérente, produit ou récupère le PDF correspondant et applique un contrôle atomique du quota. Le compteur ne doit jamais dépasser la limite, même en cas de requêtes simultanées. Une stratégie de réservation avec durée de vie et idempotence peut gérer les téléchargements interrompus ; sa sémantique exacte doit correspondre au code livré.

7.3 Erreurs et défaillances

Un téléchargement peut échouer après la réservation d'un quota, une génération PDF peut prendre du temps et une notification e-mail peut être indisponible. Ces situations nécessitent des transitions explicites (en attente, confirmé, annulé ou expiré selon l'implémentation), des opérations idempotentes et une séparation entre l'autorisation métier et les effets externes. La notification de dernier téléchargement ne doit pas compromettre la transaction principale.

7.3.1 Déroulement transactionnel du téléchargement

Étape	Responsable	Traitement
hline 1	Visiteur	Présente le lien public et demande le PDF.
2	API Fastify	Valide le lien et contrôle les conditions de distribution.
3	PostgreSQL	Arbitre atomiquement la réservation dans la limite du quota.
4	Service PDF	Produit ou récupère le document à une révision cohérente.
5	S3 privé	Conserve l'artefact PDF selon la politique de stockage.
6	API Fastify	Retourne le PDF ou un état d'indisponibilité explicite.
7	Notification (cible)	Déclenche le traitement de l'épuisement du quota, sans bloquer la transaction.

7.4 Outils, technologies et bibliothèques : rôle dans la solution

Le choix des outils découle des contraintes de collaboration en temps réel, de cohérence transactionnelle et de déploiement. Il importe de distinguer les technologies effectivement retenues, les services d'infrastructure déployés et les composants optionnels de l'architecture cible.

Technologie / outil	Rôle concret dans Collab Editor	Justification ou vigilance
React + TypeScript	Interfaces de rédaction, historique, partage, supervision	Composants réutilisables et typage côté client
Vite	Serveur de développement et compilation des assets statiques	Build destiné à S3 / CloudFront
Node.js 22	Exécution du backend JavaScript	Plateforme Elastic Beanstalk confirmée
Fastify	API HTTP, validation des requêtes et orchestration métier	Point d'entrée côté serveur
WebSocket	Transport bidirectionnel des opérations et de la présence	Connexion persistante, à sécuriser par WSS en production
ShareDB / OT	Synchronisation et transformation des éditions concurrentes	Convergence fondée sur les révisions et un serveur coordinateur
PostgreSQL	Documents, historique, droits, crédits, quotas et transactions	Source de vérité pour les invariants métier
Redis / Valkey	Coordination et messages éphémères entre composants	Ne remplace pas la persistance transactionnelle
PDFKit	Construction programmatique des documents PDF	Maîtrise du contenu et de la génération serveur
Unicode / UTF-8	Représentation et transport du texte international	Un encodage correct ne garantit pas à lui seul les glyphes PDF
Noto Sans (police Unicode)	Rendu des accents et glyphes couverts dans les PDF	Police à incorporer ; couverture selon variantes disponibles
Amazon S3	Conservation privée des PDF et hébergement cible des assets frontend	Contrôle IAM et absence d'accès public direct aux PDF
Elastic Beanstalk	Exécution et déploiement du backend	Environnement mono-instance actuellement confirmé
Amazon RDS PostgreSQL	Base relationnelle managée	TLS et accès réseau contrôlés
ElastiCache Valkey	Service de cache / coordination managé	TLS et groupes de sécurité
Secrets Manager + IAM	Gestion des secrets et des autorisations du backend	Évite d'inclure des identifiants dans Git
CloudFront (cible)	Distribution HTTPS des assets frontend et routage éventuel de l'API	Frontend publié via S3 privé et CloudFront
Git / GitHub	Historique du code, branches et collaboration de développement	Traçabilité des évolutions
AWS CLI	Provisionnement, publication et diagnostics AWS	Commandes reproductibles, permissions minimales

Unicode, UTF-8 et génération PDF : un problème distinct de la collaboration

La collaboration doit préserver exactement le contenu textuel soumis par les rédacteurs, y compris les accents, caractères non latins, signes monétaires et caractères combinants. Unicode définit les points de code des caractères ; UTF-8 est une représentation en octets utilisée pour les échanges et le stockage. Ces deux notions ne suffisent toutefois pas à assurer le rendu dans un PDF : la bibliothèque de génération doit disposer d'une police contenant les glyphes nécessaires et l'incorporer correctement dans le fichier produit.

L'utilisation de PDFKit avec une police telle que Noto Sans vise à éviter les substitutions, carrés vides ou caractères incorrects dans les PDF. La couverture exacte dépend des fichiers de police embarqués : Noto Sans standard ne couvre pas à elle seule tous les systèmes d'écriture ni les emojis. Pour une langue ou un alphabet supplémentaire, il faut choisir la variante Noto adaptée, gérer éventuellement une police de repli et valider le résultat par un PDF de test.

Les essais de validation doivent comprendre au minimum : français accentué (é, è, ç, œ), ponctuation typographique, symboles monétaires (€, £), texte multiligne et caractères internationaux effectivement exigés par le projet. Ils doivent vérifier à la fois la présence visuelle des glyphes, l'absence de troncature et la possibilité de copier le texte lorsque le générateur le permet.

Choix d'architecture et limites de l'outillage

ShareDB a été préféré à une implémentation CRDT développée pour le projet parce que la coordination centrale et les révisions serveur s'accordent avec les règles de crédit, de blocage et de persistance. PostgreSQL porte les décisions transactionnelles ; Redis sert à la coordination sans devenir une seconde source de vérité. Le choix de services AWS managés réduit l'administration système mais impose une configuration rigoureuse du réseau, des certificats TLS, des permissions IAM et de la surveillance.

Schéma complémentaire non reproduit dans cette édition ; mécanisme décrit dans le texte.

8. Déploiement et architecture AWS

8.1 Architecture cible

La cible sépare la diffusion statique React (S3 privé et CloudFront) du backend Fastify/ShareDB (Elastic Beanstalk). RDS PostgreSQL assure la persistance ; ElastiCache Valkey la coordination ; S3 privé conserve les PDF ; IAM et Secrets Manager limitent les accès. Une évolution vers plusieurs instances et un équilibreur de charge est envisageable, mais relève d'un stade distinct de l'infrastructure actuellement confirmée.

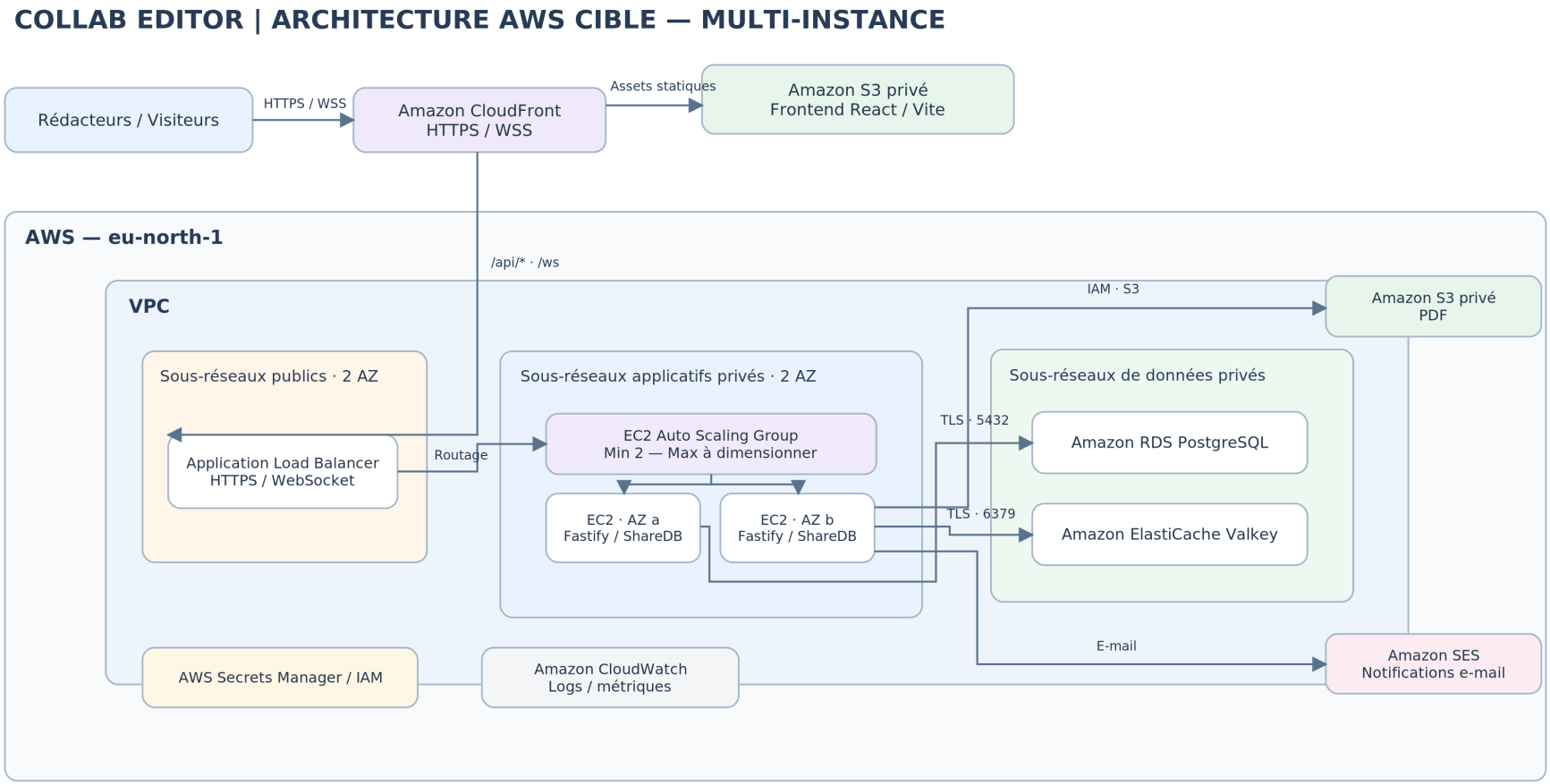


Figure 4 – Architecture AWS cible – application multi-instance et services managés (non entièrement déployée).

8.2 État de déploiement vérifié au 26 octobre 2026

Composant	État connu
Elastic Beanstalk	Environnement Collab-editor-env-1 ; Node.js 22 / Amazon Linux 2023 ; version 1.1 ; santé Ok.
Instance applicative	Une instance EC2 dans l'environnement observé ; aucune preuve d'ALB ou de seconde instance.
API	GET /health sur domaine EB HTTP : réponse HTTP 200, {"status" :"ok"}.
RDS PostgreSQL	Instance collab-editor-db configurée ; tests métier complets non encore démontrés.
ElastiCache Valkey	Ressource et accès réseau configurés ; connexion applicative complète à confirmer.
S3 PDF	Bucket privé colab-editor-pdfs configuré.
Secrets Manager	Référence de secret de connexion DB configurée côté backend.
Frontend CloudFront	Frontend React/Vite déployé sur S3 privé et distribué par CloudFront ; parcours applicatifs à distinguer des tests de charge.

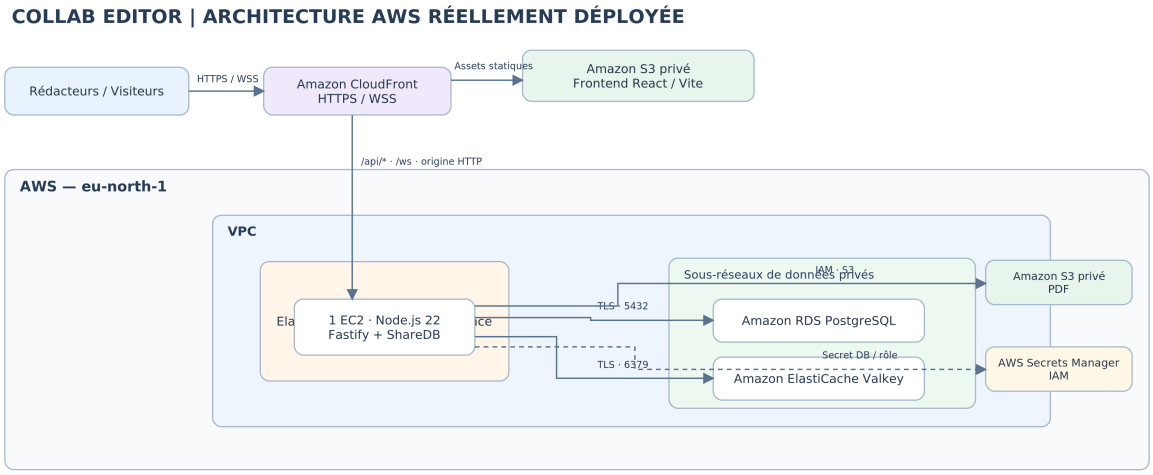


Figure 5 – Architecture AWS réellement déployée : backend mono-instance.

8.3 Difficultés rencontrées et corrections

La mise en production a nécessité plusieurs itérations de diagnostic et de correction, tant sur l'infrastructure que sur la logique métier. Les difficultés les plus significatives sont regroupées ci-dessous.

Difficulté	**Cause ou manifestation**	**Traitement et enseignement**

Déploiement Elastic Beanstalk et PostgreSQL	Les scripts de migration, les variables d'environnement et le cycle de démarrage de l'application ne se comportaient pas comme dans l'environnement local.	Adapter le processus de migration et vérifier la configuration de connexion avant de conclure au succès du déploiement.
Connexion TLS à Amazon RDS	Des erreurs de certificat ont empêché la connexion attendue à la base de données.	Corriger la chaîne de confiance et la configuration TLS ; ne pas désactiver la vérification des certificats comme solution durable.
Accès à ElastiCache Valkey	Des délais d'attente réseau ont été observés, notamment en raison des règles de groupes de sécurité.	Vérifier les flux réseau et les autorisations entre le backend et le cache.
Gestion des quotas de téléchargement	Des demandes simultanées pouvaient entrer en concurrence pour les dernières places disponibles.	Concevoir une réservation transactionnelle et des états explicites pour empêcher le dépassement du quota.
Réservation et génération du PDF	La réservation précède une génération potentiellement lente ou défaillante ; une réservation en cours ne correspond pas encore à un PDF distribué.	Distinguer réservation, génération, succès et indisponibilité temporaire ; traiter les échecs et les expirations sans double comptabilisation.
Distribution via CloudFront	Le frontend HTTPS, les chemins API et la connexion WebSocket devaient être compatibles avec l'origine backend.	Configurer le routage et vérifier séparément les parcours HTTP et WebSocket.

Lors d'un essai concurrent réalisé sur AWS, 20 demandes presque simultanées ont été envoyées pour un lien limité à 10 téléchargements : 10 PDF ont été retournés et 10 réponses HTTP 409 (*temporarily_unavailable*) ont été observées, sans onzième PDF. Le résultat soutient l'invariant de non-dépassement du quota pour ce scénario précis, mais révèle une distinction importante entre une place temporairement réservée et un quota définitivement épuisé. Il ne constitue pas une validation exhaustive des échecs de génération, des reprises ni du fonctionnement multi-instance.

Le point de santé HTTP `/health` répondait également 200 ; ce contrôle atteste la disponibilité de l'API au moment du test, sans valider à lui seul les flux applicatifs de bout en bout.

8.4 Limites et risques résiduels

L'origine Elastic Beanstalk a été testée en HTTP. Le frontend CloudFront est publié en HTTPS ; la liaison vers l'origine Elastic Beanstalk demeure en HTTP et impose un chemin HTTPS/WSS côté navigateur, éventuellement via un proxy CloudFront adapté ; le chiffrement du segment CloudFront–origine devra être renforcé pour une production durable. Les scénarios de montée en charge, de reprise après panne, de notification et de cohérence sous concurrence demandent des tests spécifiques.

9. Stratégie de validation

Famille de test	Scénario et résultat attendu
Collaboration	Deux rédacteurs modifient le même document ; convergence et absence de perte.
Persistance	Redémarrage API puis réouverture ; modifications acceptées conservées.
Crédit	Deux sessions concurrentes proches du plafond ; total accepté $\leq$ 10 000.
PDF	Modification après publication ; téléchargement suivant reflète la nouvelle révision.
Quota	N requêtes concurrentes pour quota Q ; distributions autorisées $\leq$ Q.
Panne Redis	Dégradation explicitée ; aucune acceptation mensongère d'opérations.
Sécurité	Aucun secret dans le build frontend ; S3 PDF inaccessible publiquement.
AWS	GET /health HTTP 200 vérifié ; tests WebSocket, API métier et CloudFront à compléter.

10. Discussion critique et perspectives

L’architecture répond à un compromis entre qualité de collaboration, contrôle central des règles métier et simplicité d’exploitation. L’OT via ShareDB est adaptée au périmètre actuel, mais impose de tester soigneusement la validation des opérations et les reconnexions. PostgreSQL est essentiel pour les invariants financiers au sens large – quotas et crédits – alors que Redis ne doit pas être traité comme stockage durable des éditions. L’architecture AWS offre une séparation des responsabilités, mais la résilience multi-instance, la sécurité de bout en bout et les mesures de performance doivent être démontrées avant d’être présentées comme acquises.

Conclusion

Collab Editor constitue un exercice complet de conception de système collaboratif : il relie un problème utilisateur à des choix d'algorithmes de convergence, à des transactions de cohérence et à une architecture de déploiement. L'apport du projet ne se limite donc pas à la réalisation d'une interface de texte ; il tient surtout à la formalisation et au traitement de la concurrence, des droits, de la traçabilité et des contraintes de diffusion. Les limites de validation et les écarts entre architecture cible et déploiement réel sont explicités dans les sections correspondantes.

Références et éléments à consolider

— ShareDB – documentation du framework et protocole de collaboration (référence technique à compléter avec version exacte).

— Littérature OT : travaux fondateurs sur la transformation opérationnelle et la convergence .

— Littérature CRDT : articles de synthèse sur les structures répliquées convergentes .

— Google Docs – documentation publique sur la collaboration et l'historique ; ne pas extrapoler son architecture interne.

— AWS – documentation Elastic Beanstalk, CloudFront OAC, RDS TLS, ElastiCache et Secrets Manager .

— Sources projet : migrations SQL, scripts de déploiement, tests et événements Elastic Beanstalk du 26 octobre 2026.

11. État de validation et résultats mesurés

L'application Collab Editor est accessible à l'adresse publique https://dlnopdrpomzao.cloudfront.net/. Son interface React est hébergée dans un compartiment Amazon S3 privé et distribuée aux utilisateurs par Amazon CloudFront (distribution `E3980PPRULYQTO`). Les requêtes de l'application sont transmises au backend Fastify, déployé sur une instance Amazon EC2 gérée par Elastic Beanstalk (environnement `Collab-editor-env-1`, région `eu-north-1`). Les documents et les opérations sont conservés dans PostgreSQL sur Amazon RDS ; Valkey, sur Amazon ElastiCache, prend en charge les fonctions de cache et de coordination prévues par l'application. Les fichiers PDF sont stockés dans un compartiment S3 distinct.

Les utilisateurs accèdent au site en HTTPS et aux connexions collaboratives en WSS. CloudFront relaie ensuite les communications vers l'environnement Elastic Beanstalk, dont l'origine utilise actuellement HTTP. Cette configuration correspond à la version déployée et testée. L'équilibrage de charge entre plusieurs instances, l'Auto Scaling et l'envoi de courriels via Amazon SES appartiennent à l'architecture cible présentée précédemment.

11.1 Campagne fonctionnelle Playwright

Campagne communiquée : 10 scénarios PASS, 1 FAIL (présence et curseur distant après reconnexion), 2 SKIPPED/BLOCKED (crédit quotidien). Les tests réussis ne constituent pas une preuve de capacité à 300 rédacteurs simultanés.

11.2 Concurrence des téléchargements PDF

Campagne AWS réelle : 20 demandes de téléchargement lancées sur 26 ms pour un lien limité à 10 copies. Résultat : 10 réponses 200 application/pdf, signature %PDF-, taille 11 149 octets ; 10 réponses 409 temporarily_unavailable sans PDF ; aucune erreur réseau ni timeout ; compteur final 10/10 consommés. Aucun onzième PDF n'a été observé. Le critère de refus définitif 410 exhausted pendant cette rafale n'est pas satisfait : le test est FAIL strict pour la sémantique des réponses, mais PASS pour le non-dépassement du quota observé.

11.3 Limites et perspectives

Les 15 000 visiteurs (dont 80 % en une minute), les 300 rédacteurs simultanés, la convergence ShareDB multi-instance et l'Auto Scaling n'ont pas été validés en charge. Le diagramme multi-instance illustre une architecture cible. La notification e-mail via Amazon SES est une option de conception, non une preuve d'envoi effectif.

Les journaux de tests, la documentation API, les procédures de déploiement et les relevés de coûts constituent des pièces complémentaires distinctes de ce dossier.

Annexe A – Cas d’utilisation et descriptions textuelles

La figure distingue les acteurs principaux, à gauche, du **service de messagerie**, acteur secondaire à droite. Le tableau récapitule l’ensemble des cas d’utilisation ; les fiches détaillées précisent les scénarios et les exceptions.

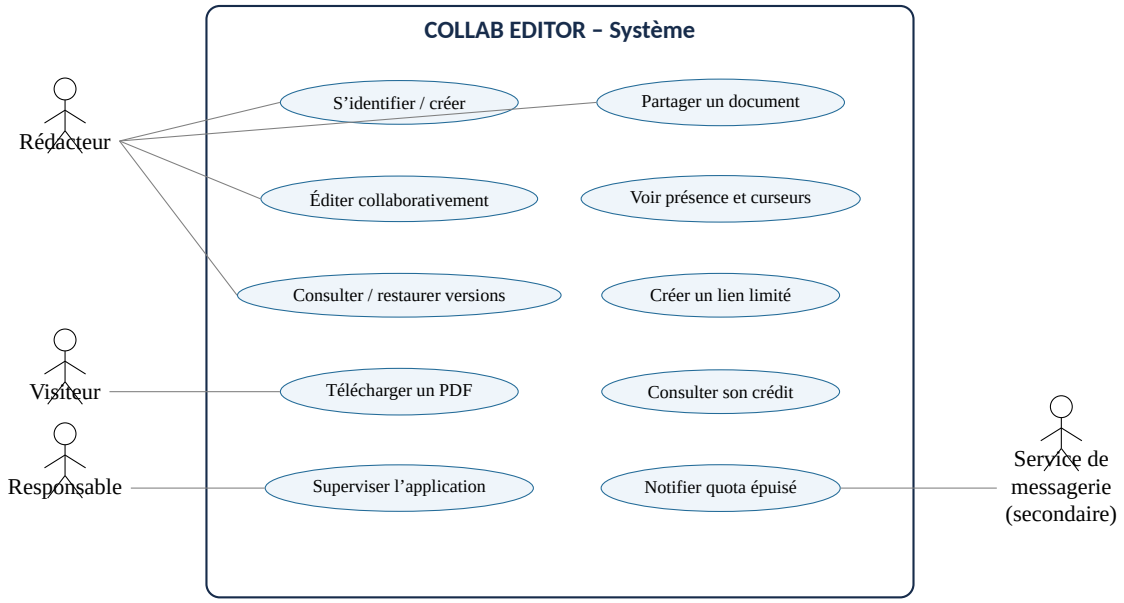


Figure 6 – Vue synthétique des interactions UML ; le service de messagerie est un acteur secondaire externe.

Tableau de couverture des cas d’utilisation

Code	Cas d’utilisation	Acteur principal
hline UC-01	S’identifier comme rédacteur	Rédacteur
UC-02	Créer un document	Rédacteur
UC-03	Partager un document	Rédacteur
UC-04	Modifier un document collaborativement	Rédacteur
UC-05	Visualiser la présence et les positions d’écriture	Rédacteur
UC-06	Consulter l’historique des versions	Rédacteur
UC-07	Restaurer une version antérieure	Rédacteur autorisé
UC-08	Créer un lien de téléchargement limité	Rédacteur autorisé
UC-09	Télécharger un document au format PDF	Visiteur
UC-10	Consulter son crédit d’écriture	Rédacteur
UC-11	Consulter l’état de l’application	Responsable
UC-12	Recevoir une notification d’épuisement du lien	Créateur du lien
hline		

UC-01 – S’identifier comme rédacteur

Rubrique	Description
hline **Élément Description**	Acteur Rédacteur
Élément Description	Objectif Accéder à l’application sous une identité déclarée
Précision	Préconditions Application accessible
Précision	Déclencheur Le rédacteur ouvre l’application
Précision	Postconditions Une session est associée à un profil de rédacteur
Étape 1	Le rédacteur ouvre l’application.
Étape 2	Le système présente un formulaire demandant son nom et son adresse e-mail.
Étape 3	Le rédacteur renseigne les informations.
Étape 4	Le système vérifie leur format.
Étape 5	Le système recherche un profil correspondant à l’adresse e-mail déclarée.
Étape 6	Si aucun profil n’existe, le système en crée un.
Étape 7	Le système ouvre une session associée au rédacteur.
Étape 8	Le rédacteur accède à son espace de travail.
Précision	* A1 : l’adresse e-mail existe déjà ; le système réutilise le profil correspondant.
Précision	* E1 : les informations sont invalides ; le système demande une correction.
Précision	* E2 : le service est indisponible ; le système indique que l’identification ne peut pas aboutir.
Précision	L’identification reste déclarative : elle ne constitue pas une vérification de la propriété de l’adresse e-mail.

UC-02 – Créer un document

Rubrique	Description
hline **Élément Description**	Acteur Rédacteur
Élément Description	Objectif Créer un document textuel
Précision	Préconditions Rédacteur identifié
Précision	Déclencheur Sélection de « Nouveau document »
Précision	Postconditions Document créé, enregistré et accessible à son créateur
Étape 1	Le rédacteur demande la création d’un document.
Étape 2	Le système présente un espace de création.

Étape 3	Le rédacteur renseigne un titre et, éventuellement, un contenu initial.
Étape 4	Le système vérifie les informations et les règles d'écriture applicables.
Étape 5	Le système crée le document et l'associe à son créateur.
Étape 6	Le système enregistre son état initial.
Étape 7	Le document est ouvert dans l'éditeur.
Précision	* A1 : le rédacteur crée un document vide.
Précision	* E1 : le contenu initial dépasse le crédit disponible ; la création avec ce contenu est refusée.
Précision	* E2 : l'enregistrement échoue ; le système n'annonce pas la création comme réussie.
Précision	Le titre est une hypothèse d'interface raisonnable, non une obligation explicite de l'énoncé.

UC-03 – Partager un document

Rubrique	**Description**
hline **Élément Description**	Acteur Rédacteur
Élément Description	Objectif Autoriser d'autres rédacteurs à accéder au document
Précision	Préconditions Document existant ; acteur autorisé à le partager
Précision	Déclencheur Sélection de « Partager »
Précision	Postconditions Les rédacteurs désignés disposent de l'accès accordé
Étape 1	Le rédacteur ouvre un document.
Étape 2	Il sélectionne la fonction de partage.
Étape 3	Le système demande les adresses e-mail des rédacteurs concernés.
Étape 4	Le rédacteur indique les destinataires.
Étape 5	Le système vérifie qu'il est autorisé à partager ce document.
Étape 6	Le système enregistre les autorisations.
Étape 7	Il confirme le partage.
Précision	* A1 : un destinataire possède déjà l'accès ; le système évite de créer un doublon.
Précision	* A2 : le destinataire n'a pas encore de profil ; son accès pourra être associé à l'adresse déclarée lors de son arrivée.
Précision	* E1 : l'acteur n'est pas autorisé à partager le document ; l'opération est refusée.
Précision	Nous devrons préciser en conception si le partage accorde systématiquement le droit d'écriture ou si plusieurs niveaux de permission sont nécessaires.

UC-04 – Modifier un document collaborativement

Rubrique	Description
hline **Description**	C’est le cas d’utilisation central du système.
Élément Description	Acteur Rédacteur
Précision	Acteurs associés Autres rédacteurs du document
Précision	Objectif Modifier un document partagé sans perdre les modifications acceptées
Précision	Préconditions Session valide, accès en écriture, rédacteur non bloqué
Précision	Déclencheur Le rédacteur ouvre un document et commence à écrire
Précision	Postconditions Les modifications acceptées sont conservées et les participants convergent vers le même état
Étape 1	Le rédacteur ouvre le document.
Étape 2	Le système lui présente son contenu courant.
Étape 3	Le système établit sa participation à la session collaborative.
Étape 4	Le rédacteur effectue une modification.
Étape 5	Le système reçoit cette modification.
Étape 6	Il vérifie les conditions d’acceptation : autorisation, blocage, crédit disponible et validité de l’opération.
Étape 7	Le système accepte et enregistre durablement la modification.
Étape 8	Il confirme l’acceptation au rédacteur.
Étape 9	Il rend la modification visible aux autres participants.
Étape 10	Les participants disposent finalement du même état du document.
Précision	* A1 – Deux rédacteurs modifient simultanément le même emplacement : le système préserve toutes les modifications acceptées et assure la convergence.
Précision	* A2 – Un rédacteur est momentanément déconnecté : à sa reconnexion, il retrouve l’état courant et le statut de ses modifications non confirmées.
Précision	* A3 – Une modification est visible localement avant sa confirmation : l’interface distingue cet état provisoire d’une modification enregistrée.
Précision	* E1 – Crédit insuffisant : modification refusée, crédit inchangé.
Précision	* E2 – Rédacteur bloqué : écriture refusée et durée du blocage indiquée.
Précision	* E3 – Accès en écriture absent : modification refusée.
Précision	* E4 – Modification invalide : opération refusée avec motif.
Précision	* E5 – Enregistrement impossible : aucune confirmation d’acceptation n’est envoyée.
Précision	Règle fondamentale : aucune modification acceptée ne doit disparaître à cause d’une autre modification concurrente. Le mécanisme qui assure cette propriété sera défini pendant la conception.

UC-05 – Visualiser la présence et les positions d’écriture

Rubrique	Description
hline **Acteur**	rédacteur ayant accès au document.
Précondition	document ouvert dans une session collaborative.
Étape 1	Le rédacteur ouvre le document.
Étape 2	Le système lui indique les autres participants actuellement présents.
Étape 3	Chaque participant est représenté par son nom ou un repère visuel.
Étape 4	Lorsqu’un participant déplace son curseur ou sa sélection, le système transmet sa nouvelle position.
Étape 5	Les autres participants voient cette position actualisée.
Étape 6	Lorsqu’un participant quitte la session, sa présence est retirée.
Alternatives	si un participant perd sa connexion, sa présence est retirée après détection de la déconnexion ; si sa position n’est pas encore connue, son nom peut être affiché sans curseur.
Postcondition	les informations de présence reflètent l’état connu des sessions, avec un délai de propagation à définir.

UC-06 – Consulter l’historique des versions

Rubrique	Description
hline **Acteur**	rédacteur autorisé.
Précondition	document existant et accessible.
Étape 1	Le rédacteur ouvre le document.
Étape 2	Il sélectionne « Historique ».
Étape 3	Le système présente les versions précédentes disponibles.
Étape 4	Le rédacteur sélectionne une version.
Étape 5	Le système affiche son contenu et ses informations associées.
Étape 6	Le document courant reste inchangé.
Exceptions	version indisponible ou accès non autorisé.
Postcondition	le rédacteur a consulté une version sans modifier le document courant.

UC-07 – Restaurer une version antérieure

Rubrique	Description
hline **Acteur**	rédacteur disposant du droit de restauration.
Préconditions	document accessible ; version antérieure existante.
Étape 1	Le rédacteur consulte l'historique.
Étape 2	Il sélectionne une version antérieure.
Étape 3	Il demande sa restauration.
Étape 4	Le système présente une confirmation.
Étape 5	Le rédacteur confirme.
Étape 6	Le système restaure le contenu sélectionné comme nouvel état courant.
Étape 7	Il conserve la traçabilité de l'opération.
Étape 8	Les participants voient le nouvel état.
Alternatives	des rédacteurs sont actifs pendant la restauration ; leurs modifications déjà acceptées doivent rester traçables, et les opérations concurrentes doivent être traitées selon une règle explicite.
Exceptions	restauration non autorisée, version inexistante ou opération impossible.
Postcondition	un nouvel état courant est établi sans effacer l'historique.
Précision	Il faudra préciser en conception que restaurer un contenu ancien ne signifie pas supprimer définitivement les modifications intervenues depuis.

UC-08 – Créer un lien de téléchargement limité

Rubrique	Description
hline **Acteur**	rédacteur autorisé.
Précondition	document existant.
Étape 1	Le rédacteur ouvre un document.
Étape 2	Il demande la création d'un lien de téléchargement.
Étape 3	Le système lui demande le nombre maximal de téléchargements.
Étape 4	Le rédacteur saisit un entier positif.
Étape 5	Le système valide la demande.
Étape 6	Il crée un lien associé au document et au quota demandé.
Étape 7	Le système présente le lien au rédacteur.
Étape 8	Le rédacteur peut le communiquer aux visiteurs.
Alternatives	plusieurs liens indépendants peuvent être créés pour un même document.
Exceptions	quota invalide, accès non autorisé ou échec de création.

Postcondition	le lien existe avec un quota initial connu et un compteur de consommation nul.
Précision	Le lien reste associé au document vivant : il ne représente pas une version figée au moment de sa création.

UC-09 – Télécharger un document au format PDF

Rubrique	**Description**
hline **Acteur**	visiteur anonyme.
Précondition	possession d'un lien de téléchargement.
Étape 1	Le visiteur ouvre le lien.
Étape 2	Le système identifie le document et le quota associés.
Étape 3	Il vérifie si une distribution peut encore être autorisée.
Étape 4	Il réserve une possibilité de téléchargement selon les règles de distribution.
Étape 5	Il obtient une version complète et cohérente du document courant.
Étape 6	Il prépare le PDF correspondant.
Étape 7	Le système met le fichier à disposition du visiteur.
Étape 8	Il actualise l'état du lien.
Étape 9	Si le dernier exemplaire est consommé, le système déclenche la notification au créateur.
Précision	* A1 – La distribution doit patienter : le visiteur reçoit un état d'attente explicite.
Précision	* A2 – Le document a été modifié depuis la création du lien : le PDF correspond à son état courant au moment de référence de la distribution.
Précision	* A3 – Plusieurs visiteurs arrivent simultanément : les demandes sont traitées sans dépasser le quota global.
Précision	* E1 – Lien inexistant : information d'indisponibilité.
Précision	* E2 – Quota épuisé : aucun nouveau téléchargement autorisé.
Précision	* E3 – Erreur de génération du PDF : la demande n'est pas présentée comme réussie.
Précision	* E4 – Connexion interrompue : traitement selon la politique de réservation et de consommation à définir.
Postcondition	soit une distribution est autorisée et comptabilisée conformément au contrat, soit le visiteur connaît la raison de son indisponibilité.
Précision	Le traitement exact des téléchargements interrompus constitue une question de conception à résoudre.

UC-10 – Consulter son crédit d’écriture

Rubrique	Description
hline **Acteur**	rédacteur identifié.
Étape 1	Le rédacteur ouvre son espace de travail.
Étape 2	Il consulte son crédit disponible.
Étape 3	Le système affiche le crédit restant pour la journée.
Étape 4	Il indique, si nécessaire, que le crédit est épuisé.
Postcondition	aucune modification du crédit.
Précision	Ce cas est une amélioration proposée, cohérente avec la limite quotidienne imposée.

UC-11 – Consulter l’état de l’application

Rubrique	Description
hline **Acteur**	responsable de l’association.
Précondition	accès autorisé à la supervision.
Étape 1	Le responsable ouvre l’espace de supervision.
Étape 2	Le système présente les documents actuellement ouverts.
Étape 3	Il affiche les liens actifs et leurs téléchargements restants.
Étape 4	Il affiche les rédacteurs temporairement bloqués.
Étape 5	Le responsable peut actualiser les informations.
Étape 6	Le système lui présente l’état le plus récent disponible.
Alternatives	aucune session active, aucun lien actif ou aucun rédacteur bloqué ; le système affiche des listes vides explicites.
Exception	accès non autorisé.
Postcondition	l’état du système n’est pas modifié.

UC-12 – Recevoir une notification d’épuisement du lien

Rubrique	Description
hline **Acteur bénéficiaire**	créateur du lien.
Acteur secondaire	service de messagerie.
Déclencheur	consommation du dernier téléchargement autorisé.
Étape 1	Le système constate que le quota du lien vient d’être épuisé.

Étape 2	Il identifie le créateur du lien.
Étape 3	Il prépare une notification.
Étape 4	Il transmet la notification au service de messagerie.
Étape 5	Le service de messagerie prend en charge l'envoi de l'e-mail.
Étape 6	Le système conserve l'état de la notification.
Alternatives	si l'envoi échoue temporairement, le système peut réessayer sans produire plusieurs notifications logiques pour le même épuisement.
Postcondition	l'événement de fin de distribution est enregistré et la notification est prise en charge ou reste à traiter.
Précision	Nous ne devons pas confondre la demande d'envoi d'un e-mail avec la preuve qu'il est effectivement arrivé dans la boîte de réception.

Annexe B – Technologies utilisées et justification des choix

Technologie	Rôle	Justification et limites
hline React, TypeScript, Vite	Interface web	Composants réutilisables, typage statique et compilation rapide ; la qualité collaborative dépend toutefois du backend.
Node.js 22, Fastify	API HTTP et WebSocket	Modèle événementiel adapté aux connexions simultanées ; nécessite une discipline de gestion des erreurs et de l'état partagé.
ShareDB / OT	Synchronisation documentaire	Transforme les opérations concurrentes pour viser la convergence ; n'assure pas à lui seul la cohérence sémantique du texte.
PostgreSQL (Amazon RDS)	Persistance transactionnelle	Transactions et contraintes adaptées aux révisions, autorisations et quotas ; exige des accès atomiques.
Valkey (ElastiCache)	Cache et coordination	Faible latence, gestion d'états temporaires ; l'intégration inter-instance ShareDB doit être vérifiée avant tout Auto Scaling.
Amazon S3	Stockage d'objets	Conservation privée des PDF et hébergement des fichiers statiques ; accès encadrés par IAM et politiques de compartiment.
Amazon CloudFront	Distribution et point d'entrée HTTPS	Cache du frontend et routage des chemins API/WebSocket ; configuration des origines et invalidations à maîtriser.
AWS Elastic Beanstalk	Exécution du backend	Déploiement géré et environnement reproductible ; le déploiement observé reste mono-instance.
Amazon SES (cible)	Notification e-mail	Service managé envisageable pour l'alerte de quota épuisé ; déploiement effectif non confirmé.
ALB + Auto Scaling (cible)	Répartition et élasticité	Architecture résiliente envisagée ; non déployée et non validée dans cette version.
Playwright	Tests de bout en bout	Automatisation de parcours utilisateur ; les scénarios non passés ne sont pas considérés validés.
hline		

Annexe C – Limites, cohérence sémantique et perspectives

Cohérence syntaxique et convergence. La transformation opérationnelle (OT) vise à faire converger les répliques vers un même contenu après acceptation des opérations. Cette propriété n'implique pas que le texte résultant conserve une signification cohérente. Deux modifications techniquement compatibles peuvent produire une phrase contradictoire, une répétition ou une rupture argumentative. Une relecture éditoriale humaine demeure nécessaire.

Cohérence transactionnelle. Les quotas PDF et les crédits d'écriture exigent des invariants persistants ; le succès d'un test ponctuel ne démontre pas l'absence de toute condition de concurrence.

Limites de validation. Les tests disponibles concernent principalement le déploiement mono-instance. La synchronisation inter-instance, l'Auto Scaling, la charge cible et la réception effective des notifications par e-mail ne sont pas établis.

Limites de sécurité. L'identification déclarative par nom et e-mail ne vérifie pas la propriété de l'adresse. Elle ne remplace pas une authentification forte.

Perspectives. Valider la coordination ShareDB entre instances, compléter les tests de reconnexion et de charge, préciser les garanties de livraison des notifications et documenter le traitement des interruptions de téléchargement.

Conclusion générale

Collab Editor fournit une base technique structurée pour la rédaction collaborative d'un journal étudiant. La solution articule une interface web, un moteur d'édition concurrente, une persistance transactionnelle et des services AWS managés. La distinction entre architecture cible et architecture réellement déployée permet de présenter les résultats sans surestimer les validations. Le principal enjeu restant dépasse la simple convergence des caractères : la cohérence sémantique des contenus produits demeure une responsabilité éditoriale. Les prochaines étapes concernent la validation distribuée, les scénarios de défaillance et la montée en charge mesurée.